# 第九章 Swing图形用户界面 & 第十章 多线程

面向对象的软件构造导论

谢沛璋

# 目录


## 第九章 Swing图形用户界面

1. Swing框架
2. Swing图形处理、绘制颜色的原理
3. Swing基本用户组件
4. 事件处理机制
5. MVC模式

## 第十章 多线程

1. 进程与线程的区别
2. Java中对线程的控制
3. 同步、死锁及如何避免
4. 任务与线程池
5. 多线程应用——生产者与消费者模式

# 第九章 Swing图形用户界面

# Swing框架

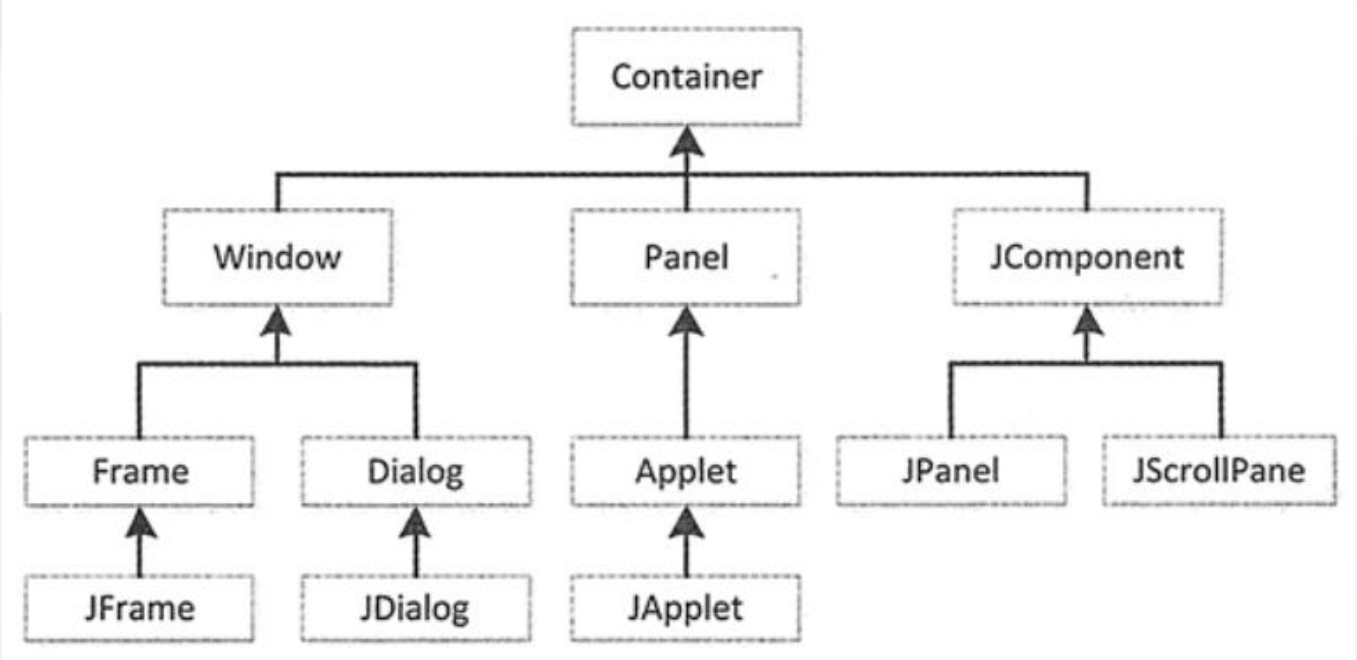
- Swing是用于Java GUI编程（图形界面设计）的工具包，采用MVC模式，实现GUI组件的显示逻辑和数据逻辑的分离。
- Swing 的两种元素：组件和容器。组件是单独的控制元素，例如按键或者文本编辑框。组件要放到容器中才能显示出来。一个容器容纳一组组件容器，本身也是组件，因此容器也可放到别的容器中。组件和容器构成了包含层级关系。





# Swing框架

- Swing的组件继承于JComponent类。
- 容器的分类：重量级：Jframe，Jdialog等，作为顶层容器，最大特点是不能被别的容器包含，只能作为界面程序的最顶层容器来包含其它组件；轻量级：作为中间容器，继承于JComponent，因此它们本身也是组件，可以（也必须）包含在其它的容器中。



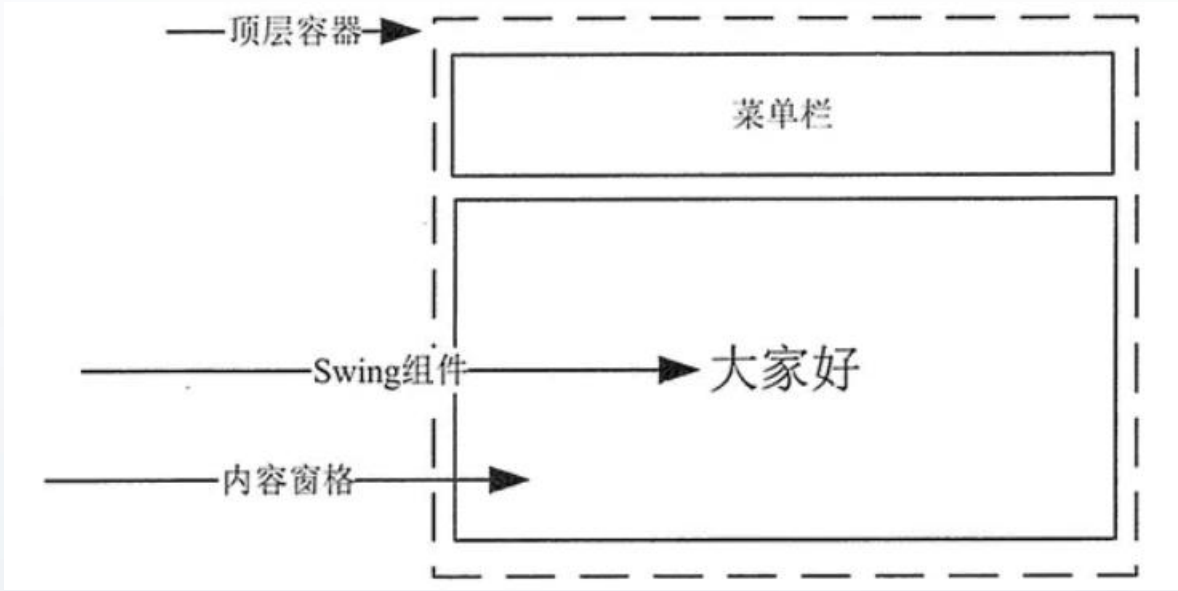
# Swing框架

- 在使用Swing向容器添加组件时，需要考虑组件的位置和大小。
- 布局管理器控制着容器中组件的位置。当向容器中增加组件时，需要给容器设置一种布局管理器，让它来管理容器中各个组件的位置，即排列布局方式。

| 布局管理器 | 特性 |
| --- | --- |
| FlowLayout | 流式布局管理器，是从左到右，中间放置，一行放不下就换到另外一行。 |
| BorderLayout | 边框布局管理器分为东、南、西、北、中心五个方位。 |
| GridLayout<br>GridBagLayout | 网格式布局。<br>网格式布局，可以放置不同大小的组件。 |
| BoxLayout | 盒布局管理器，把组件水平或者竖直排在一起。 |
| SpringLayout | 按照一定的约束条件来组织组件。 |

# Swing图形处理、绘制颜色的原理

- JFrame 是Swing组件的顶层容器，用来设计类似于 Windows 系统中窗口形式的界面，由用户的窗口系统绘制。
- 其他组件并不能够直接放到Jframe对象上面，需要将组件添加至内容窗格，内容窗格再放到 JFrame 顶层容器的上面。内容窗格是一个透明的没有边框的中间容器。
- 添加到窗体的所有组件都会自动添加到内容窗格中。



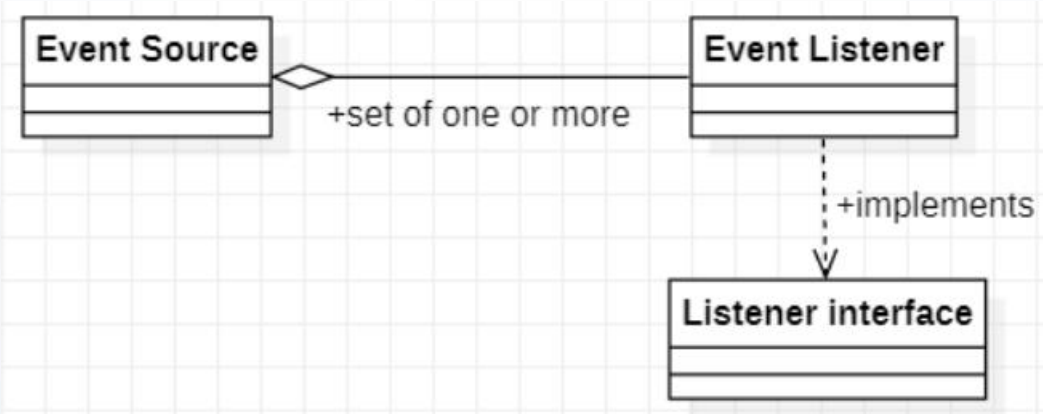
# Swing基本用户组件

- 文本输入组件（继承自JTextComponent）：
  - 文本域(JTextField)：接收单行文本
  - 文本区(JTextArea)：接收多行文本
  - 密码域(JPasswordField)：接收单行文本，且不显示文本内容

- 选择组件：
  - 复选框 JCheckBox
  - 单选按钮 ButtonGroup,JRadioButton
  - 组合框 faceCombo
  - 菜单 JMenuBar

# 事件处理机制

- 事件：用户对组件的一次操作称为一个事件，如点击一个按钮，移动鼠标，通过键盘输入一个字符，从列表中选择一个项目等。
- 事件源：事件发生的场所，通常就是各个组件如按钮或滚动条。
- 事件监听器：实现了监听器接口的类实例。
- 一个事件源可以有多个监听器。 当事件发生时，事件源将事件对象发送给所有注册的监听器。监听器对象再使用事件对象中的信息决定如何对事件做出响应。

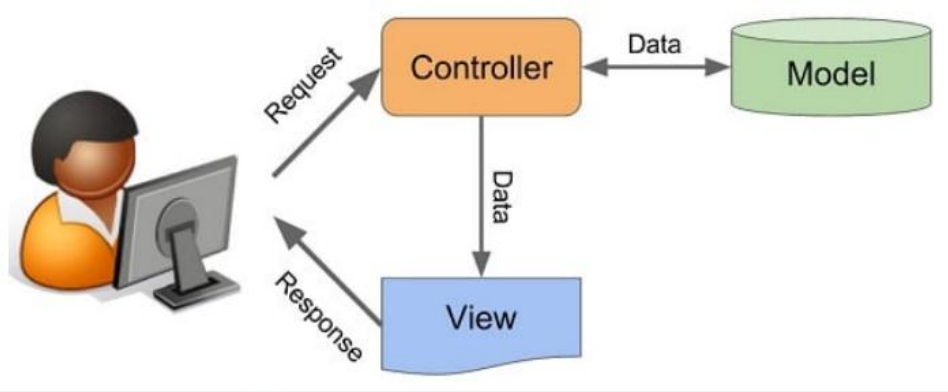


# MVC模式

- 每个组件（对象）都有三个特征 ：
  - 内容：按钮是否被按下；文本域中的文本等
  - 外观：颜色、大小等
  - 行为：对事件的反应

# MVC模式

- 为了实现这些需求，Swing采用了一种设计模式：模型-视图-控制器模式（model-view-controller, MVC）
- 模型（model）:操作内容（对应行为）。负责存储完整的内容，实现改变内容和查找内容的方法。模型没有用户界面，是完全不可见的。
- 视图（view）：显示内容（对应外观）。一个模型可以有多个视图，每个视图可以显示全部内容的不同部分或不同方面。 当模型更新时，需要通知与之关联的所有视图同步更新。
- 控制器（controller）:处理用户输入（对应内容）。负责处理用户输入事件，使视图(显示逻辑)与模型（数据逻辑）分离开，且决定是否将事件转化成对模型或视图的更改。



# MVC模式

- 特点：
  - 优点：封装变化，减少耦合。
  - 缺点：增加额外的时间和空间开销。

# 第十章 多线程

# 进程与线程的区别

- 进程是正在运行的程序的实例，线程是进程中一个单一顺序的控制流。



# 进程与线程的区别

| 进程（Process） | 线程（Thread） |
| --- | --- |
| 重量级 | 轻量级 |
| 一个应用可以包含多个进程 | 一个进程可以包含多个线程 |
| 多个进程间 不共享内存 | 一个进程的多个线程间 共享内存 |
| 进程表现为 虚拟机 | 线程表现为 虚拟CPU |
| **资源分配的最小单位** | **程序执行的最小单位** |

# Java中对线程的控制

- 线程的两种建立方式（无返回值，不可抛出异常）：
  - 继承Thread类
  - 实现Runnable接口

- 其中，Runnable更常用，其优势在于：
  - 避免由于单继承局限所带来的影响；
  - 任务与运行机制解耦，降低开销；
  - 更容易实现多线程资源共享。

```java
public class Thread1 extends Thread {
    @Override
    public void run() {
        System.out.println("New Thread");
    }
}
```

2

```java
public class Thread2 implements Runnable {
    @Override
    public void run() {
        System.out.println("New Thread");
    }
}

public class Main {
    public static void main(String[] args){
        new Thread1().start();
        new Thread(new Thread2()).start();
    }
}
```

# Java中对线程的控制

- Callable接口
- 之前提到的Runnable接口的run方法没有参数和返回值，在运算结束后不能直接返回结果。但我们经常会希望线程结束时返回运算结果，或者能抛出异常，这时就需要用到Callable。
- 使用Callable需实现call方法，并使用FutureTask控制Callable任务的执行。

```
// 实例化 Callable 任务，指定返回类型为 String
Callable<String> callable = new Callable<String>() {
    public String call() throws Exception {
        // balabalabala~
        return "Hello world ~";
    }
};

// 将 callable 任务委派给 FutureTask
FutureTask<String> task = new FutureTask<String>(callable);

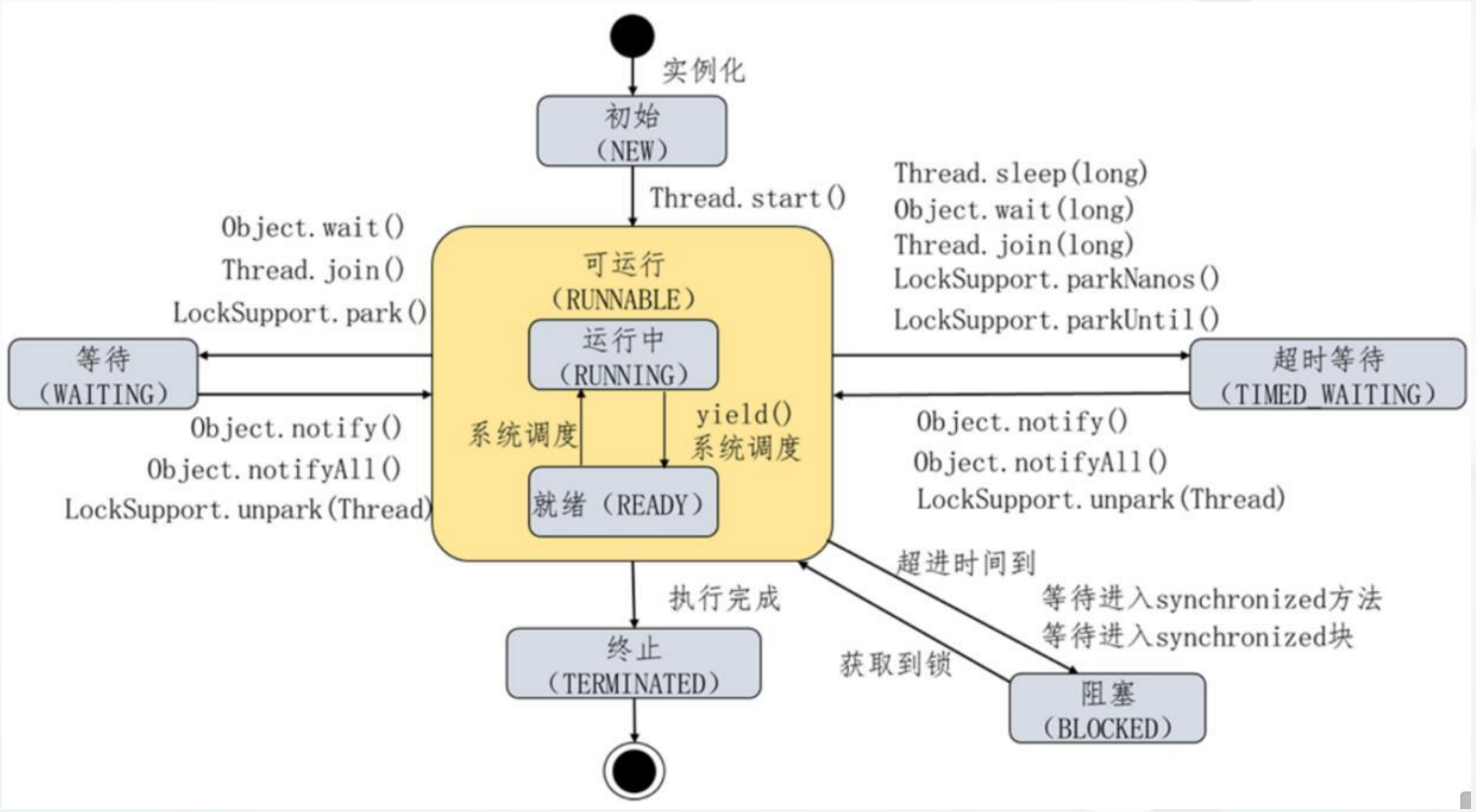
// FutureTask实现了Runnable接口，所以可以直接传给 Thread 启动线程
new Thread(task).start();

// 通过 FutureTask 获取返回值
String call = task.get();

System.out.println(call); // print: Hello world ~
```

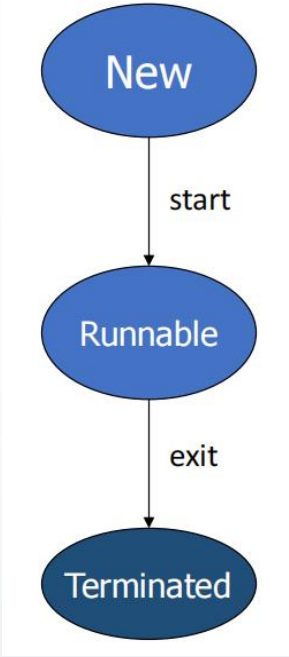
# Java中对线程的控制

- 线程一共有六种状态：初始状态，可运行状态，阻塞状态，等待状态，计时/超时等待状态和终止状态



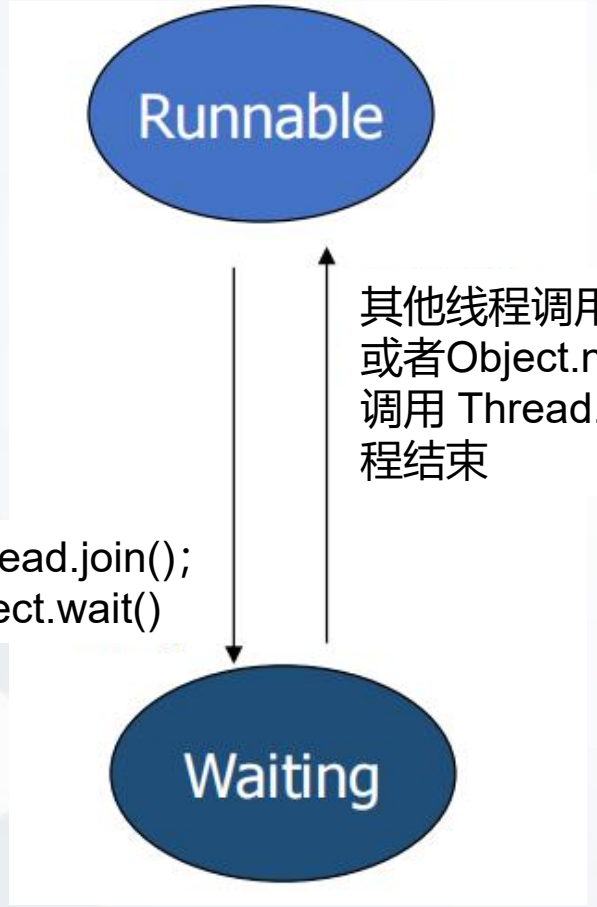
# Java中对线程的控制

- 线程的正确退出方法是使用正确的退出方法是使用interrupt()方法中断线程。该方法：
  - 其作用是中断此线程（此线程不一定是当前线程，而是指调用该方法的Thread实例所代表的线程），但实际上只是给线程设置一个中断标志，线程仍会继续运行。
  - 因此每个线程都应该不时地检查此标志（isInterrupted()来获取打断标志），以判断线程是否被中断，并自行决定决定如何响应中断。



# Java中对线程的控制

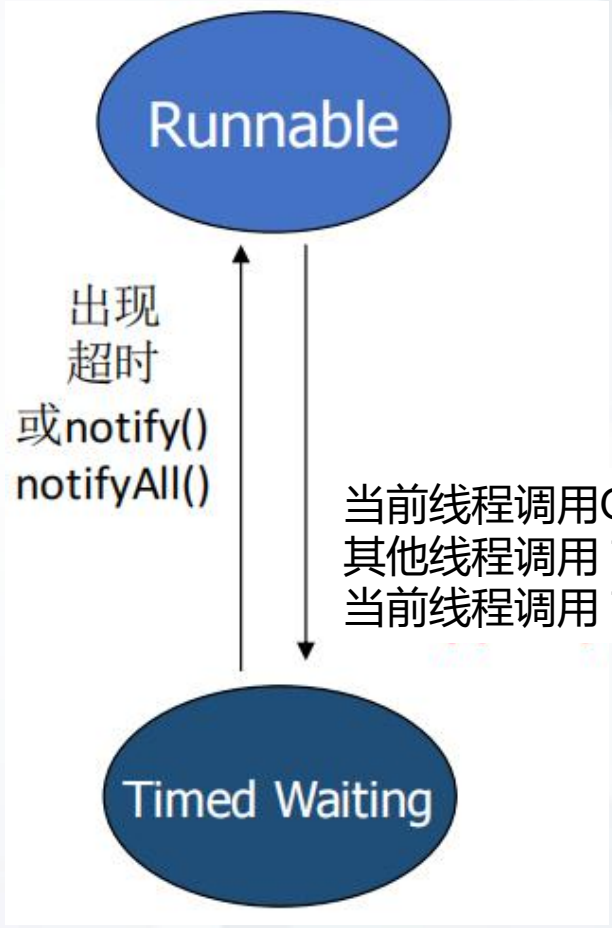
## 等待状态




其他线程调用Object.notify()
或者Object.notifyAll()
调用 Thread.join()方法的线程结束

其他线程调用 Thread.join();
当前线程调用Object.wait()

## 计时等待状态




出现
超时
或notify()
notifyAll()

当前线程调用Object.wait(time);
其他线程调用 Thread.join(time);
当前线程调用 Thread.sleep(time)

# Java中对线程的控制

- 区分Thread类和Object中的线程相关方法
  - sleep方法是让当前线程休眠，让出cpu，但是不释放锁，这是Thread的静态方法
  - wait方法是让当前线程等待，释放锁，这是Object的方法

主要总结Thread类中的核心方法

| 方法名称 | 是否static | 方法说明 |
| --- | --- | --- |
| start() | 否 | 让线程启动，进入就绪状态,等待cpu分配时间片 |
| run() | 否 | 重写Runnable接口的方法,线程获取到cpu时间片时执行的具体逻辑 |
| yield() | 是 | 线程的礼让，使得获取到cpu时间片的线程进入就绪状态，重新争抢时间片 |
| sleep(time) | 是 | 线程休眠固定时间，进入阻塞状态，休眠时间完成后重新争抢时间片,休眠可被打断 |
| join()/join(time) | 否 | 调用线程对象的join方法，调用者线程进入阻塞,等待线程对象执行完或者到达指定时间才恢复，重新争抢时间片 |
| **isInterrupted()** | 否 | 获取线程的打断标记，true:被打断，false：没有被打断。调用后不会修改打断标记 |
| interrupt() | 否 | 打断线程，抛出InterruptedException异常的方法均可被打断，但是打断后不会修改打断标记，正常执行的线程被打断后会修改打断标记 |
| **interrupted()** | 否 | 获取线程的打断标记。调用后会清空打断标记 |
| currentThread() | 是 | 获取当前线程 |

Object中与线程相关方法

| 方法名称 | 方法说明 |
| --- | --- |
| wait()/wait(long timeout) | 获取到锁的线程进入阻塞状态 |
| notify() | 随机唤醒被wait()的一个线程 |
| notifyAll(); | 唤醒被wait()的所有线程，重新争抢时间片 |

# Java中对线程的控制

- 守护线程
  - 守护线程的唯一用途是为其他线程提供服务，例如计时线程。
  - 在JVM中，所有非守护线程都执行完毕后，无论有没有守护线程，虚拟机都会自动退出。
  - 由于守护线程的终止是自身无法控制的，因此千万不要把IO、File等重要操作逻辑分配给它。

# 同步、死锁及如何避免

- 为什么需要线程同步？
  - 当多个线程同时运行时，线程的调度由操作系统决定，程序本身无法决定。因此，任何一个线程都有可能在任何指令处被操作系统暂停，然后在某个时间段后继续执行。
  - 如果多个线程同时读写共享变量，会出现数据不一致的问题。

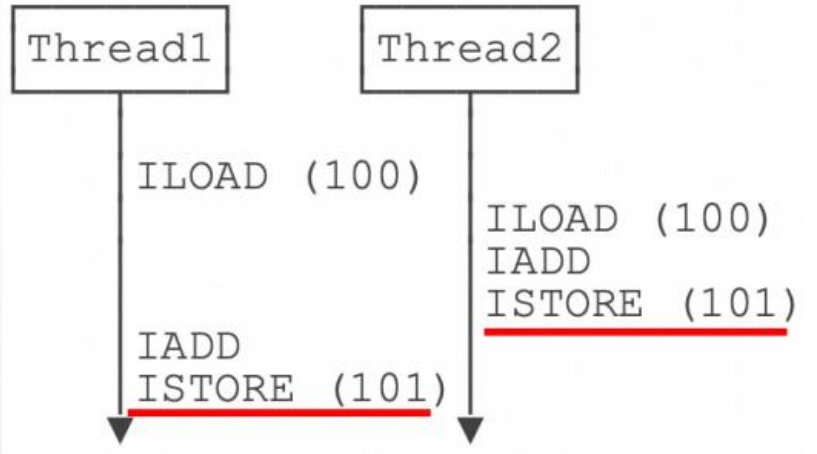
```
class Counter {
    public static int count = 0;
}

class AddThread extends Thread {
    public void run() {
        for (int i=0; i<100000; i++) { Counter.count += 1;
}
    }
}
```

```
class DecThread extends Thread {
    public void run() {
        for (int i=0; i<100000; i++) { Counter.count -= 1; }
    }
}
```

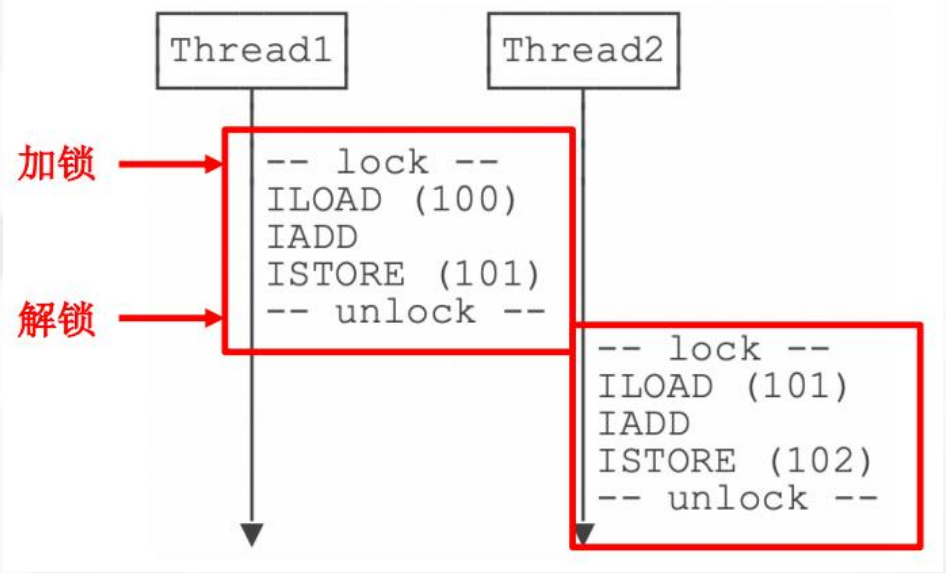
# 同步、死锁及如何避免

- 怎么实现同步？
- 首先了解原子操作：指不能被中断的一个或一系列操作。即要么完全执行，要么完全不执行，不存在执行了一半的情况。
- 例子：n = n + 1；看上去是一行语句，实际上对应了3条原子指令：
- ILOAD 取数；IADD 加法运算；ISTORE 结果保存
- 当两个线程同时执行n = n + 1时：



# 同步、死锁及如何避免

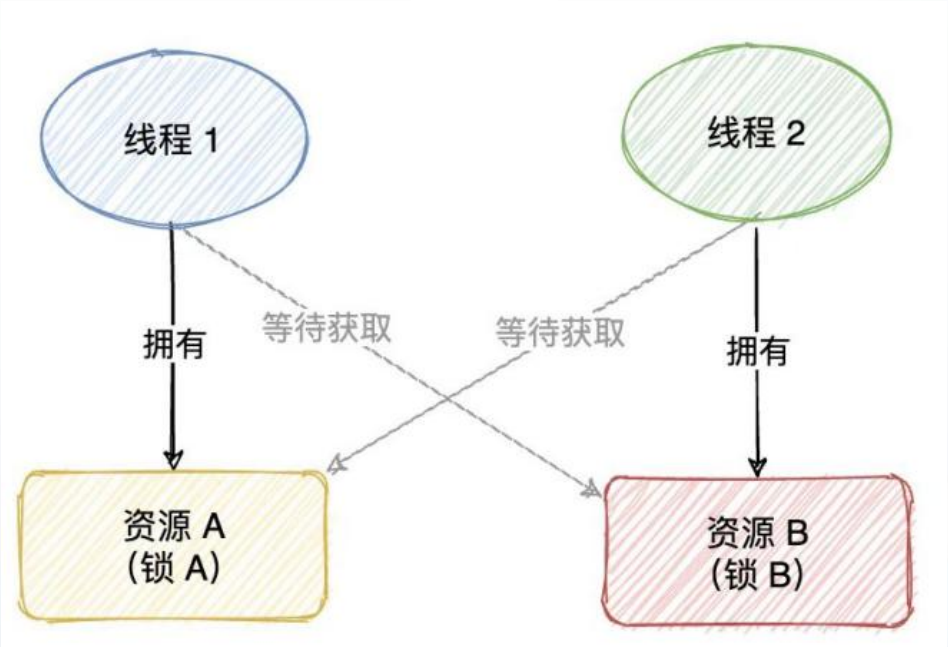
- 怎么实现同步？
- 这说明多线程模型下，要保证逻辑正确，对共享变量进行读写时，必须保证一组指令以原子方式执行：即某一个线程执行时，其他线程必须等待。
- 通过加锁和解锁的操作，保证3条指令总是在一个线程执行期间，不会有其他线程会进入此指令区间。




```
synchronized(lock) {
    n = n + 1;
}
```

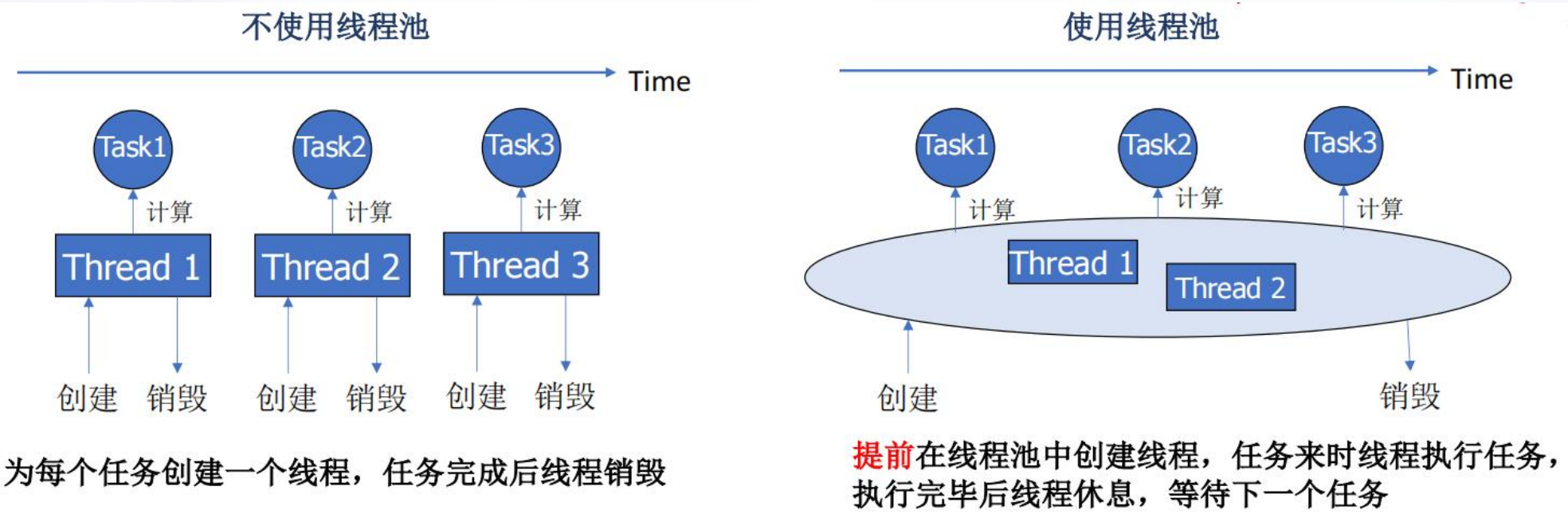
# 同步、死锁及如何避免

- 什么情况下会产生死锁
- 死锁:在线程A持有饭并想获得钱的同时，线程B持有钱并尝试获得饭，那么这两个线程将永远地等待下去。
- 避免死锁：线程获取锁的顺序要一致。即严格按照先获取钱，再获取饭的顺序，或者先获取饭，再获取钱的顺序。



# 任务与线程池

- 线程池
- 频繁创建/终止大量线程，会带来大量开销，怎么办？引入线程池。
- 利用池化技术去减少资源对象的创建次数，提高程序的性能，特别是在高并发下这种提高更加明显。

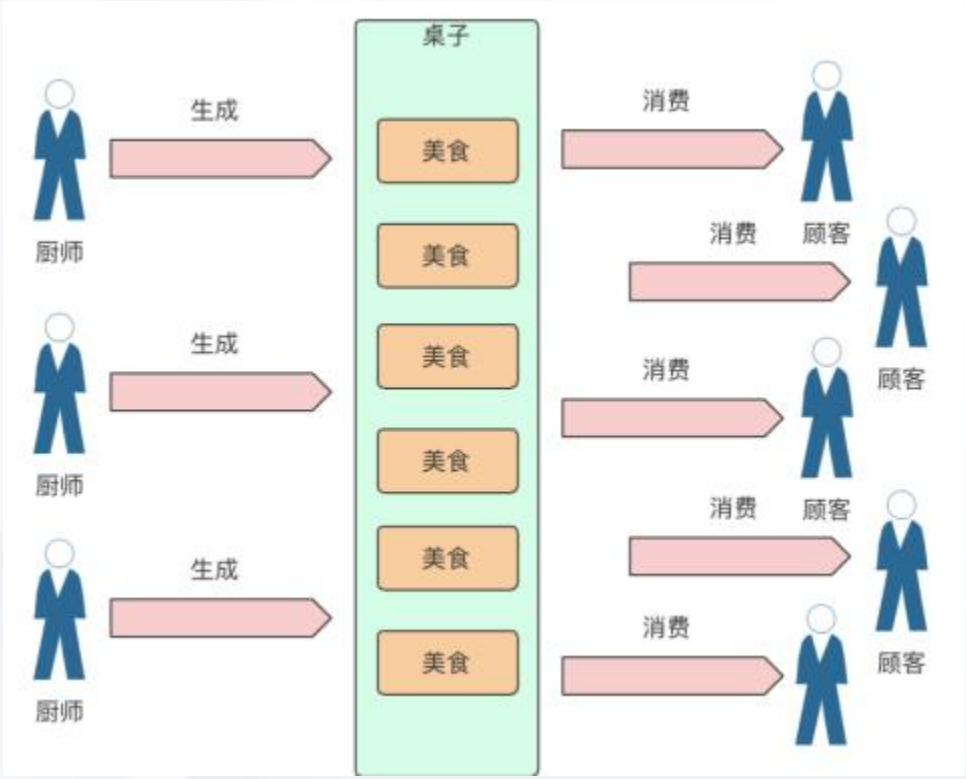


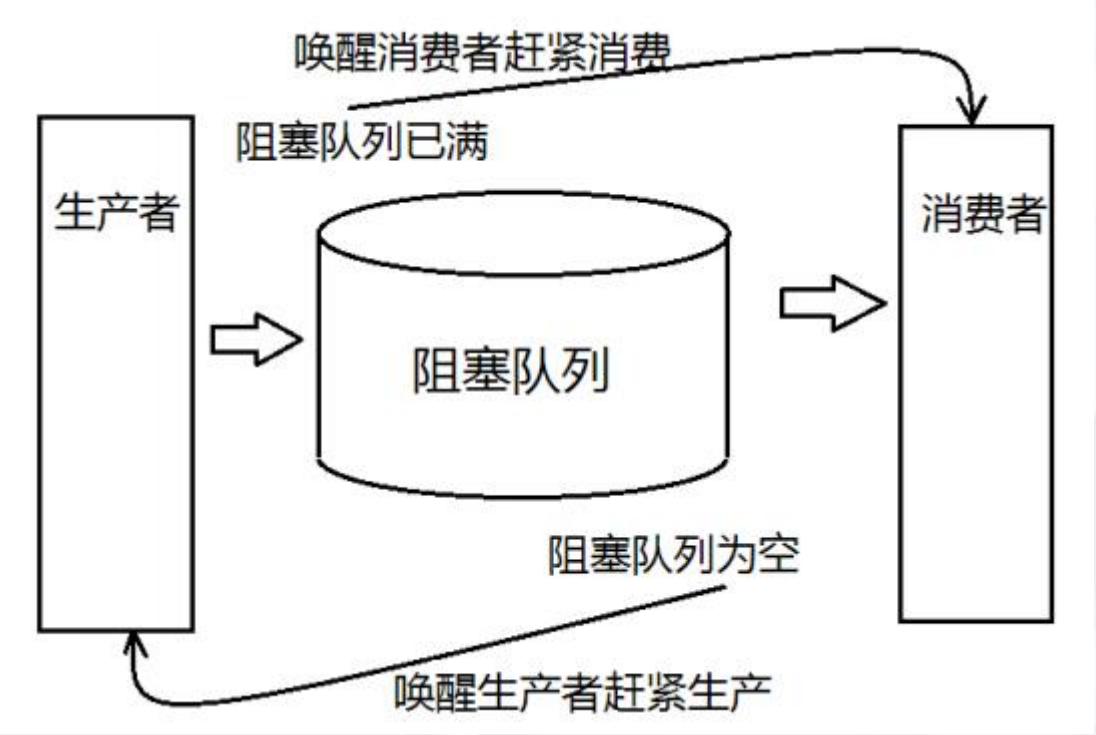
# 任务与线程池

- 使用线程池的优点：
- 降低资源消耗：通过重复利用已创建的线程降低线程创建和销毁造成的消耗。
- 提高响应速度：当任务到达时，任务可以不需要等到线程创建就能立即执行。
- 提高线程的可管理性：线程是稀缺资源，如果无限制的创建，不仅会消耗系统资源，还会降低系统的稳定性，使用线程池可以进行统一的分配、调度和监控。

# 多线程应用——生产者与消费者模式

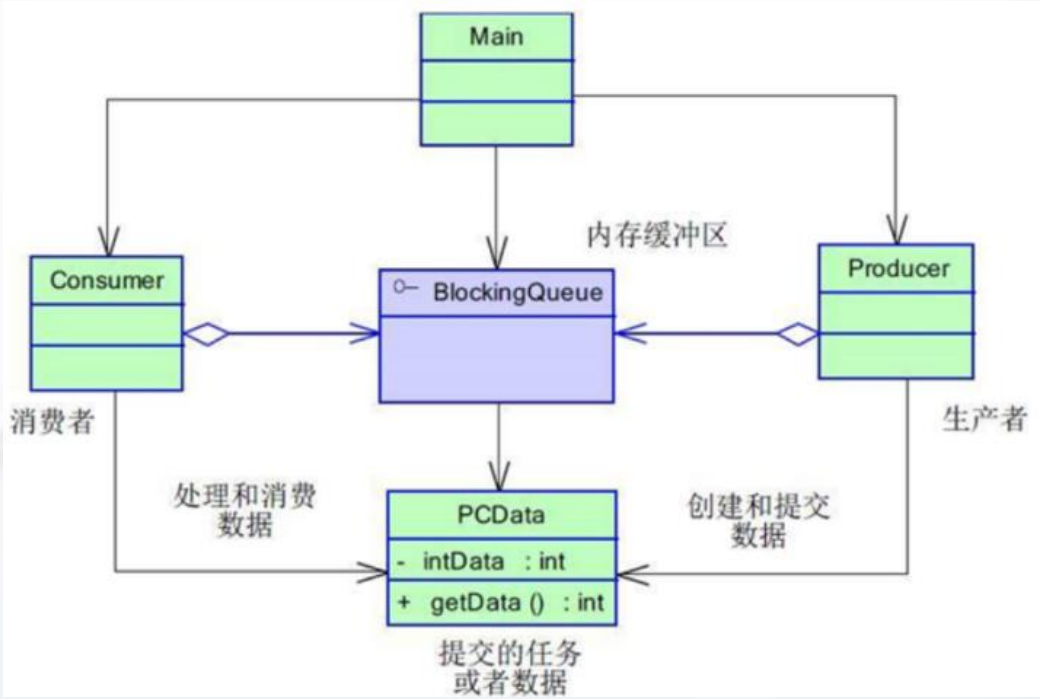
- 在实际的软件开发过程中，经常会碰到如下场景：某个模块负责产生数据，这些数据由另一个模块来负责处理。产生数据的模块，就形象地称为生产者；而处理数据的模块，就称为消费者。而生产者和消费者之间存在一个缓存区。



# 多线程应用——生产者与消费者模式

- 好处：
  - 并发（异步）：生产者和消费者各司其职，生产者和消费者都只需要关心缓冲区，不需要互相关注，通过异步的方式支持高并发，将一个耗时的流程拆成生产和消费两个阶段。
  - 解耦：生产者和消费者进行解耦（通过缓冲区通讯）。



# 多线程应用——生产者与消费者模式

Java代码实现缓冲区

```
public class Buffer {
    private List<Integer> data = new ArrayList<>();
    private static final int MAX = 2;
    private static final int MIN = 0;
    public void put(int value){
        while (true){
            try {//模拟生产数据
                Thread.sleep(500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            synchronized (this){
//当容器满的时候，producer处于等待状态
                while (data.size() == MAX){
                    System.out.println("buffer is full,waiting ....");
                    try {
                        wait();
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
//没有满，则继续produce
                System.out.println("producer--"+
Thread.currentThread().getName()+"--put:" + value);
                data.add(value);
//唤醒其他所有处于wait()的线程，包括消费者和生产者
                notifyAll();
            }
        }
    }
```

- 缓冲区的判断条件是while(condition)，防止线程被错误唤醒。
- wait()方法要放在同步块中，防止线程无法唤醒。

# 谢谢